

processed. (*END* block executes, likewise, after all the input is read and done with).

We are using *awk* to just produce the output—we don't need any input, and we discard any, if it is given.

It generates the line `1\n2\n...\n1000` and terminates...



Note: *Seq* is another command that could be used instead of *awk*, but *awk* is more ubiquitous (and I love *awk*).

For Bash and zsh people, the following incantation:

```
for ((i = 1; i < 10; ++i)); do echo "$i"; done
```

...will also work (and would not start a subprocess—so it may be faster).

The *paste* command pastes lines from two or more files together. However, in our case, which is *paste -s -d ""*, *-s* tells it to paste lines from the same file—here it is standard input—and put "" as the delimiter in between.

The *awk* command output, `1\n2\n3...\n1000` goes as input to *paste*. *Paste* dutifully puts a "*", the multiplication operator in between the two numbers. This is all the transformation it does and writes the output, *unknowingly*, to *bc*...

Here, by design, we avoid appending one "" too many. *Paste* really pastes things, in between—so it's convenient, simpler and, hence, bug-free.

bc is a very powerful calculator. It does not suffer from the overflow limitations. Why? It uses string algorithms to compute the incoming computation.

The mighty UNIX/Linux horizon is dotted with many little languages. *bc* is one such language.

Now if the result is a very big line, *bc* normally prints it pretty well.

Now if the result is a very big line, *bc* breaks it into multiple lines and puts a backslash, the line continuation character, at the end of each line (except the last). This would interfere with our future processing, down the pipeline. So we use another time honoured tradition in UNIX to configure the program behaviour with an environment variable.

That is why we need *BC_LINE_LENGTH=10000*. This stops *bc* from thinking the line as too big and thus doesn't break it

into many smaller lines—so no extraneous backslashes.

The third line in my signature is:

```
perl -pe 'a/.*?(0+$/length($1))/e'
```

Here, we feed the *bc* result to Perl. The above line uses two Perl RE specialities:

1. Non-greedy qualifiers
2. Evaluating code as part of substitution RHS

`/.*?(0+$/length($1))/e` makes it start from the beginning, and keeps scanning and discarding chars till it gets one or more trailing zeroes, and remembers this *trailing zero* string in the special variable `$1`. So, for example, given the following string: `"34311224323000000000000000000230000"`, `$1` will hold `"0000"`.

The `.*?` part modifies the `.*` behaviour to work in a non-greedy fashion—the idea is to give preference to the part that comes after `?`—so `.*?` will stop consuming as soon as the part after `?` matches. You can imagine the engine looking to the right of the `.*?`; and if the right side matches, prefer it (that is, try to match it). [`.*` matches in a greedy fashion—gobble up (almost) everything—and stop being too greedy only if the entire match fails.]

Note that the LHS matches all of the string, so we are reading and throwing away the input, and replacing it with the length of `$1`, which holds the trailing zeroes string.

Normally, you would say the substitution RHS is a string, but here a little Perl magic helps—the RHS could be a small Perl program/snippet which would be run, and its output is the real RHS.

And so we get the following answer.

There are 249 zeros in the number generated :-)

Try it out buddies—the Linux command line is a lot of fun. I thank my stars I discovered it at the beginning of my career. Penguins are beautiful :-)



By: Atul Shrinivas Khot

The author works at IBM Software Labs in Hyderabad as a senior software engineer, and has been dabbling with UNIX/Linux for the last 14 years. He is into Java/J2EE, Groovy and Ruby these days, but at times he hankers after dear-old C++ and Perl. He loves design patterns, algorithms, multi-threading and refactoring code to make it stylish. And of course, he loves vim (never miss a chance to use it) and zsh. He collects classic British mysteries (especially those green and white Penguin crime classics—penguins make life so delectable :-))

Enterprise Mail Server, Linux SBS Server,
Anti SPAM, Antivirus and HTTP Filtering.

Bandwidth Management, Internet Access Control,
Content Filtering, Web Access Reporting.

Customised Linux Product Development.

TechnoInfotech™

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Tel.: 91-22-6633 8900 Ext. 324. info@technoinfotech.com